

1 Computer System Overview

Main Elements and Registers

Processor	executes instructions and controls system operation.
Main memory	stores currently used programs and data.
I/O modules	move data between the computer and external devices.
System bus	carries data, address, and control signals among modules.

User-visible registers: referenced by machine instructions to reduce memory access. Control/status registers: PC, IR, MAR, MBR, PSW; used by CPU and OS to control execution.

Instruction Actions

- Processor-memory: load/store between CPU and memory.
- Processor-I/O: transfer data between CPU and I/O modules.
- Data processing: arithmetic or logical operations.
- Control: branch, jump, call, return; changes execution order.

Interrupts

Purpose and Types

Interrupt: a mechanism by which I/O, timer, program error, or hardware failure interrupts normal processor execution. Main purpose: improve processor utilization.

Program	divide by zero, overflow, illegal instruction.
Timer	lets the OS regain control periodically.
I/O	device reports completion or readiness.
Hardware failure	power failure, memory parity error.

Processing Sequence

1. Device issues interrupt; CPU finishes current instruction.
2. CPU acknowledges, saves PC and PSW, and loads ISR address.
3. ISR saves remaining state, handles the event, restores state.
4. Old PC and PSW are restored; interrupted program resumes.

PC tells where to resume; PSW records processor state. Polling repeatedly tests device flags; interrupts let the device notify the CPU.

Multiple Interrupts

Sequential: disable interrupts during ISR; new interrupts stay pending. Simple but may delay urgent events.

Nested: assign priorities; a higher-priority interrupt may interrupt a lower-priority ISR.

Memory, Cache, I/O

Memory Hierarchy

Higher levels are faster, smaller, and more expensive per bit; lower levels are larger and slower. Distinguish levels by capacity, access time, cost per bit, and frequency of access.

$$T_{avg} = \sum_i \left(\prod_{j < i} (1 - H_j) \right) T_i$$
$$C_{avg} = \sum C_i S_i / \sum S_i$$

Cache and Locality

Cache: small fast memory between processor and main memory; stores recently or frequently used blocks to reduce average access time.

Temporal locality: a referenced item is likely to be referenced again soon; keep recent items in fast memory.

Spatial locality: nearby addresses are likely to be referenced soon; fetch a block, not only one word.

I/O and DMA

Programmed I/O: CPU waits by polling flags such as FGI/FGO.

Interrupt-driven I/O: IEN enables the device to interrupt when ready.

DMA: transfers data directly between I/O device and memory; DMA gets high memory priority because device delay may cause data loss, while CPU delay usually only slows execution.

Chapter 1 Exam Questions

Review 1.1–1.10

Q: List the four main computer elements.
A: Processor, main memory, I/O modules, system bus.
Q: Define the two register categories.
A: User-visible registers; control and status registers.
Q: What is an interrupt?
A: A mechanism that lets another module interrupt normal CPU execution so an ISR can handle the event.
Q: How are multiple interrupts handled?

A: Sequential processing disables interrupts during ISR; nested processing uses priorities.

Q: What distinguishes memory hierarchy levels?

A: Capacity, access time, cost per bit, and access frequency.

Q: Multiprocessor vs multicore?

A: Multiprocessor has multiple processors; multicore has multiple cores on one chip.

Problem Stems and Answers

1.1 I/O program	Load dev5, add M[940], store dev6: dev6 receives 0005. Expand Fig. 1.4 execution: fetch via MAR/MBR; final M[941] = 0005. 24-bit address field gives 16 MB; PC = 24 bits, IR = 32 bits. 16-bit memory: 128 KiB; 8-bit memory: 64 KiB; 8-bit port number: 256 ports. 8 MHz, 16-bit bus, 4 clocks/bus cycle gives 4 MB/s; wider bus or doubled clock gives 8 MB/s. FGI/FGO polling wastes CPU; IEN enables interrupt-driven I/O. DMA gets memory priority because device delay may lose data; CPU delay mostly slows execution. 9600 bps DMA slows 1 MIPS CPU by about 0.12%. Programmed I/O = 25,000 words/s; DMA = 2.15×10^6 words/s. Spatial: sequential a[i]; temporal: repeated same a[i] in inner loop. Use $T_{avg} = \sum_i (\prod_{j < i} (1 - H_j)) T_i$, $C_{avg} = \sum C_i S_i / \sum S_i$. Main memory $\approx$ \$83.89; cache-tech $\approx$ \$838.86; $H \approx 99.17\%$. Average access time $\approx$ 480,026 ns.
1.2 MAR/MBR	
1.3 32-bit instr.	
1.4 16-bit addr.	
1.5 bus rate	
1.6 Teletype I/O	
1.7 DMA priority	
1.8 DMA slowdown	
1.9 I/O rate	
1.10 locality code	
1.11 n-level memory	
1.12 cost/hit	
1.13 cache/disk	
1.14 stack PC	No; the PC cannot generally be eliminated by using the stack top.

2.1 OS Objectives and Functions

Definition

Operating system: a program that controls application execution and acts as an interface between applications and computer hardware.

Three Objectives

Convenience	makes the computer easier to use.
Efficiency	uses CPU, memory, I/O, files, storage, and network resources efficiently.
Evolution	permits new functions, tests, fixes, and hardware support without disrupting service.

Two Main Roles

- User/computer interface: hides hardware details and provides usable services.
- Resource manager: allocates processor, memory, I/O devices, files, storage, and network resources.

OS Services

Program development	editors, debuggers, compilers, libraries.
Program execution	loads code/data, initializes files, I/O, and resources.
I/O access	uniform device interface, e.g. read(), write().
File access	file organization, permissions, protection.
System access	login, authorization, resource contention control.
Error handling	detect hardware/software errors; terminate, retry, or report.
Accounting	collect usage/performance data; support billing and tuning.

Interfaces: ISA vs ABI vs API

ISA	CPU instruction set	hardware/software boundary; user ISA vs system ISA.
ABI	binary compatibility	system-call interface, registers, calling convention, binary format.
API	source-level calls	library functions used by programmers, e.g. open(), malloc().

Memory hook: API = coding; ABI = compiled binary execution; ISA = CPU instructions.

Kernel, Control, Evolution

Kernel / Nucleus

Kernel: central OS part, normally resident in main memory, containing the most frequently used OS functions: process, memory, I/O, file, interrupt, and system-call handling.

How the OS Controls the CPU

The OS is also software. It gains control, schedules and allocates resources, gives control to user programs, then regains control through interrupts, system calls, or exceptions.

Why OSs Must Evolve

- Hardware upgrades: new processors/devices require new support.
- New services: users and administrators need new tools and features.
- Fixes: bugs must be repaired without destabilizing old services.

System Calls and Modes

System Calls

System call: controlled entry point for a user program to request OS services such as file I/O, process creation, memory allocation, communication, and protection.

Dual-Mode Operation

User mode	low privilege; applications cannot directly execute privileged instructions.
Kernel mode	high privilege; OS kernel performs protected operations.

Flow: a user program requests a system call/trap; the kernel performs the service and returns to user mode.

PDF Stems and Answers

- R2.1 Stem: three OS-design objectives. A: convenience, efficiency, evolution.
- R2.2 Stem: kernel of an OS. A: memory-resident core with frequent OS functions.
- P2.4 Stem: system-call purpose and dual-mode relation. A: safe service request; user mode traps to kernel mode, then returns.
- P2.5 Stem: OS/390 SRM checks untouched page frames. A: find inactive pages, guide replacement/reallocation, detect memory pressure or thrashing.
- R2.6 Stem: five storage duties. A: isolation, allocation, modular support, protection/access control, long-term storage.
-

Fast Recall

- OS = application controller + hardware interface + resource manager.
- OS hides hardware details behind services and interfaces.
- Important services: development, execution, I/O, files, access control, errors, accounting.
- Kernel holds common privileged OS functions in memory.
- System calls are the legal bridge from user mode to kernel mode.
- Evolution needs modular design, clear interfaces, and maintainability.